Thapathali Campus
Institute of Engineering, Tribhuvan University

Computing Eigenvalues and Eigenvectors

Using the Inverse Power Method

Subject: Numerical Analysis

Topic: Inverse Power Method with Python Implementation

Covering: Smallest, Largest, and Intermediate Eigenvalues

Name	:	Srijan Pokhrel
Program	:	MSC in Informatics and Intelligent System Engineering
Campus	:	Thapathali Campus
Roll Number	:	THA082MSISE017
Subject	:	Numerical Analysis
Topic	:	Inverse Power Method
Language	:	Python 3 with NumPy

Contents

1. Introduction

Eigenvalues and eigenvectors are among the most important concepts in linear algebra and numerical analysis. They appear in virtually every area of science and engineering, from structural mechanics to data science.

For large matrices, computing eigenvalues analytically is impractical. **Iterative numerical methods** are therefore used. This document focuses on the three most important iterative techniques:

- **Power Method** – finds the *largest* eigenvalue
- **Inverse Power Method** – finds the *smallest* eigenvalue
- **Shifted Inverse Power Method** – finds any *intermediate* eigenvalue

1. Mathematical theory of eigenvalues and eigenvectors
2. Step-by-step algorithm for each method
3. Complete flowchart of the Inverse Power Method
4. Full Python implementation with verified output
5. Comparison of what each method reveals
6. Real-world applications

2. Mathematical Background

2.1 Eigenvalues and Eigenvectors

Definition 2.1. *Let A be an $n \times n$ square matrix. A scalar λ is called an **eigenvalue** of A if there exists a non-zero vector $\mathbf{x}$ such that:*

$$A\,\mathbf{x} = \lambda\,\mathbf{x}$$

*The vector $\mathbf{x} \neq 0$ is called the **eigenvector** corresponding to λ.*

This equation says: when matrix A acts on the special vector $\mathbf{x}$, it only *scales* it by the factor λ, without changing its direction.

2.2 Characteristic Equation

Rearranging $A\mathbf{x} = \lambda\mathbf{x}$:

$$(A - \lambda I)\,\mathbf{x} = \mathbf{0}$$

For a non-trivial solution ($\mathbf{x} \neq \mathbf{0}$) to exist, the matrix $(A - \lambda I)$ must be singular:

$$\det(A - \lambda I) = 0$$

Solving this polynomial of degree n gives the n eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$.

2.3 Types of Eigenvalues

Once all eigenvalues are ordered by magnitude $|\lambda_1| \leq |\lambda_2| \leq \cdots \leq |\lambda_n|$, they can be classified:

Type	Description	*Notation*
Largest	Greatest magnitude; dominant mode of the system	$\lambda_n = \lambda_{\max}$
Smallest	Least magnitude; most stable mode	$\lambda_1 = \lambda_{\min}$
Intermediate	All eigenvalues between smallest and largest	$\lambda_2, \ldots, \lambda_{n-1}$

2.4 Physical Meaning of Each Eigenvalue

- Represents the **dominant mode** of a system
- Controls how fast a dynamic system grows or oscillates
- Used for: principal stress analysis, dominant vibration frequency, largest singular value in data analysis
- Found using: **Power Method**

- Represents the **most stable or least dominant mode**
- Determines the minimum load a structure can carry before buckling
- Used for: minimum natural frequency, critical buckling load, stability bounds, ground state energy in quantum mechanics
- Found using: **Inverse Power Method**

- Any eigenvalue between the smallest and largest
- Used for: specific resonance modes, structural analysis at a particular frequency, targeted data compression
- Found using: **Shifted Inverse Power Method** (choose shift σ close to the target eigenvalue)

3. Algorithms

3.1 Power Method – Largest Eigenvalue

The Power Method finds $\lambda_{\max}$ by repeated multiplication with A.

1. **Initialize:** Choose $\mathbf{x}^{(0)} = [1, 1, \ldots, 1]^T$
2. **Multiply:** $\mathbf{y}^{(k)} = A\,\mathbf{x}^{(k-1)}$
3. **Find scale:** $m^{(k)} = \max\left(\left|\mathbf{y}^{(k)}\right|\right)$
4. **Normalize:** $\mathbf{x}^{(k)} = \dfrac{\mathbf{y}^{(k)}}{m^{(k)}}$
5. **Check convergence:** $\left\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\right\| < \varepsilon \quad \Rightarrow$ Stop
6. Else: $k \leftarrow k + 1$ and go to Step 2

Result: $\lambda_{\max} \approx m^{(k)}$, eigenvector $\approx \mathbf{x}^{(k)}$

3.2 Inverse Power Method – Smallest Eigenvalue

Key Idea: The eigenvalues of A^{-1} are the *reciprocals* of the eigenvalues of A:

$$A\mathbf{x} = \lambda\mathbf{x} \implies A^{-1}\mathbf{x} = \tfrac{1}{\lambda}\mathbf{x}$$

So the *largest* eigenvalue of A^{-1} corresponds to the *smallest* eigenvalue of A. We apply the Power Method to A^{-1}.

1. **Initialize:** Choose $\mathbf{x}^{(0)} = [1, 1, \ldots, 1]^T$
2. **Compute:** A^{-1} (inverse of matrix A)
3. **Multiply:** $\mathbf{y}^{(k)} = A^{-1}\mathbf{x}^{(k-1)}$
4. **Find scale:** $m^{(k)} = \max\left(\left|\mathbf{y}^{(k)}\right|\right)$
5. **Normalize:** $\mathbf{x}^{(k)} = \dfrac{\mathbf{y}^{(k)}}{m^{(k)}}$
6. **Check convergence:** $\left\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\right\| < \varepsilon \quad \Rightarrow$ Stop
7. Else: $k \leftarrow k + 1$ and go to Step 3

Result: $\lambda_{\min} = \dfrac{1}{m^{(k)}}$, eigenvector $\approx \mathbf{x}^{(k)}$

3.3 Shifted Inverse Power Method – Intermediate Eigenvalue

Key Idea: If we shift the matrix by σI, the eigenvalues of $(A - \sigma I)$ become $(\lambda_i - \sigma)$. The smallest shifted eigenvalue corresponds to the eigenvalue of A *closest to* σ.

$$(A - \sigma I)\,\mathbf{x} = (\lambda - \sigma)\,\mathbf{x}$$

Apply Inverse Power Method to $(A - \sigma I)$ to find the eigenvalue λ closest to the shift σ:

$$\lambda = \sigma + \frac{1}{m^{(k)}}$$

1. **Choose shift:** Select σ close to the target eigenvalue (but not exactly equal to any eigenvalue)
2. **Compute:** $B = A - \sigma I$
3. **Compute:** B^{-1} (inverse of shifted matrix)
4. **Multiply:** $\mathbf{y}^{(k)} = B^{-1}\,\mathbf{x}^{(k-1)}$
5. **Find scale:** $m^{(k)} = \max\left(\left|\mathbf{y}^{(k)}\right|\right)$
6. **Normalize:** $\mathbf{x}^{(k)} = \dfrac{\mathbf{y}^{(k)}}{m^{(k)}}$
7. **Check convergence:** $\left\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\right\| < \varepsilon \quad \Rightarrow$ Stop
8. Else: $k \leftarrow k + 1$ and go to Step 4

Result: $\lambda_{\text{target}} = \sigma + \dfrac{1}{m^{(k)}}$, eigenvector $\approx \mathbf{x}^{(k)}$

4. Flowchart of the Inverse Power Method

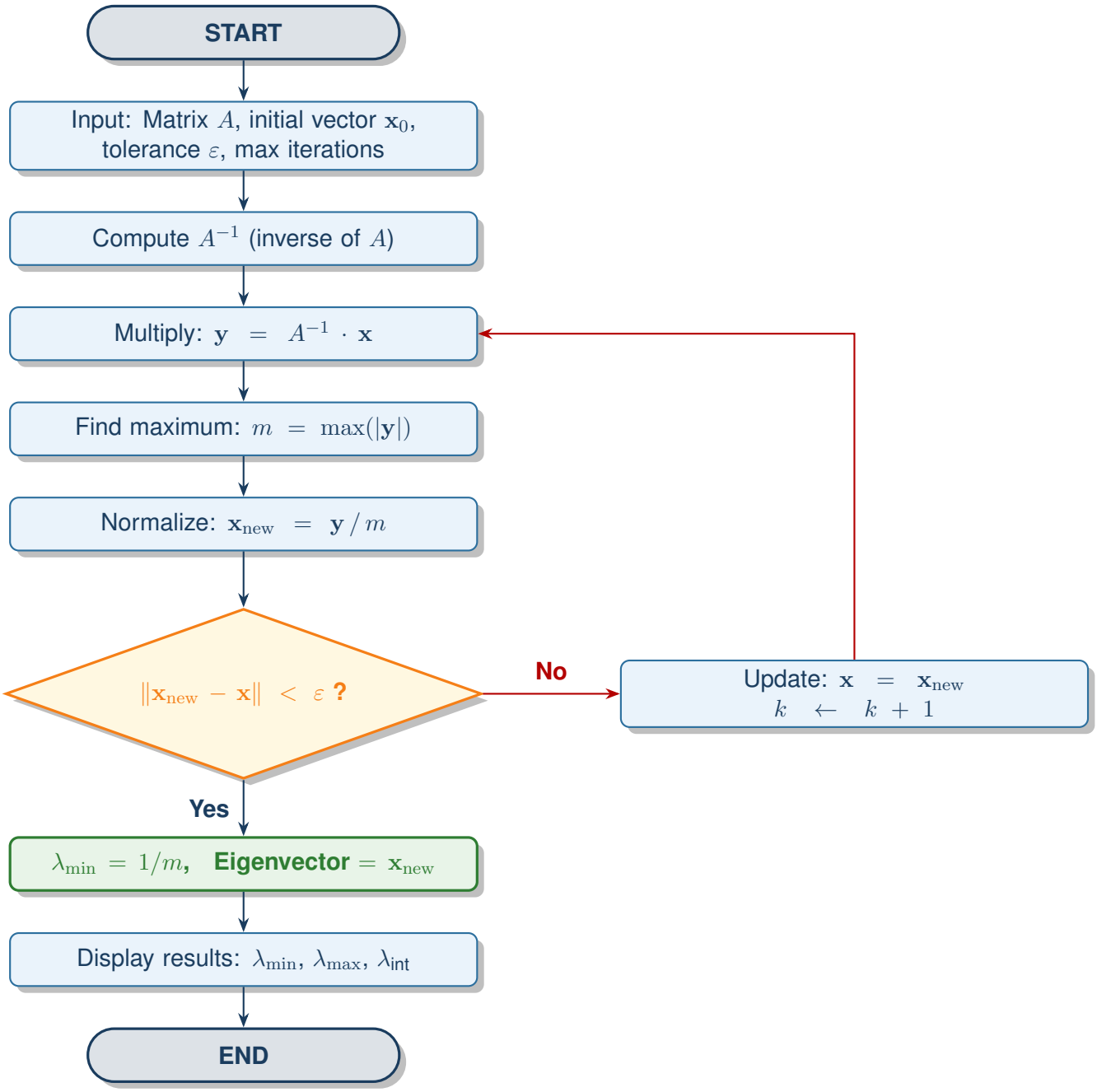

Figure 1: Flowchart of the Inverse Power Method for finding the smallest eigenvalue

5. Worked Example

5.1 Matrix Setup

Consider the following 3×3 symmetric matrix:

$$A = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

The initial vector is $\mathbf{x}^{(0)} = [1,\ 1,\ 1]^T$ and tolerance $\varepsilon = 10^{-6}$.

5.2 First Iteration (Inverse Power Method)

Step 1: Compute A^{-1}:

$$A^{-1} = \frac{1}{\det(A)} \begin{bmatrix} 5 & -2 & 1 \\ -2 & 8 & -4 \\ 1 & -4 & 11 \end{bmatrix} \quad \text{where } \det(A) = 18$$

Step 2: $\mathbf{y}^{(1)} = A^{-1}\mathbf{x}^{(0)}$

$$\mathbf{y}^{(1)} = \frac{1}{18} \begin{bmatrix} 5-2+1 \\ -2+8-4 \\ 1-4+11 \end{bmatrix} = \frac{1}{18} \begin{bmatrix} 4 \\ 2 \\ 8 \end{bmatrix} = \begin{bmatrix} 0.222 \\ 0.111 \\ 0.444 \end{bmatrix}$$

Step 3: $m^{(1)} = \max(|0.222,\ 0.111,\ 0.444|) = 0.444$

Step 4: Normalize: $\mathbf{x}^{(1)} = \begin{bmatrix} 0.500 \\ 0.250 \\ 1.000 \end{bmatrix}$

Iterate until convergence. The method converges to:

$$\lambda_{\min} = \frac{1}{m^{(\infty)}} \approx 1.267950 \qquad \mathbf{x}_{\min} \approx \begin{bmatrix} 0.2679 \\ -0.7321 \\ 1.0000 \end{bmatrix}$$

5.3 Summary of All Results

Method	Eigenvalue Found	Value
Power Method	Largest ($\lambda_{\max}$)	4.732051
Inverse Power Method	Smallest ($\lambda_{\min}$)	1.267950
Shifted (shift=2.5)	Intermediate (λ_{int})	3.000000
NumPy Verification	All eigenvalues (sorted)	$1.2679,\ 3.0000,\ 4.7321$

6. Python Implementation

6.1 Complete Code

```python
import numpy as np

# ------------------------------------------------
# POWER METHOD -- Finds the LARGEST Eigenvalue
# ------------------------------------------------
def power_method(A, tol=1e-6, max_iter=1000):
    n = A.shape[0]
    x = np.ones(n)                          # Initial vector
    for i in range(max_iter):
        x_new = A @ x                       # Matrix-vector multiply
        eigenvalue = np.max(np.abs(x_new))
        x_new = x_new / eigenvalue          # Normalize
        if np.linalg.norm(x_new - x) < tol:
            return eigenvalue, x_new        # Converged
        x = x_new
    return eigenvalue, x


# ------------------------------------------------
# INVERSE POWER METHOD -- Finds the SMALLEST Eigenvalue
# ------------------------------------------------
def inverse_power_method(A, tol=1e-6, max_iter=1000):
    n = A.shape[0]
    x = np.ones(n)                          # Initial vector
    A_inv = np.linalg.inv(A)                # Compute A inverse
    for i in range(max_iter):
        x_new = A_inv @ x                   # Multiply by A inverse
        m = np.max(np.abs(x_new))
        x_new = x_new / m                   # Normalize
        if np.linalg.norm(x_new - x) < tol:
            return 1.0 / m, x_new           # Smallest = 1 / largest of A_inv
        x = x_new
    return 1.0 / m, x


# ------------------------------------------------
# SHIFTED INVERSE POWER METHOD -- Finds INTERMEDIATE Eigenvalue
# Note: shift must NOT equal any eigenvalue of A
# ------------------------------------------------
def shifted_inverse_power_method(A, shift, tol=1e-6, max_iter=1000):
    n = A.shape[0]
    I = np.eye(n)
    x = np.ones(n)                          # Initial vector
    A_shifted_inv = np.linalg.inv(A - shift * I)
    for i in range(max_iter):
        x_new = A_shifted_inv @ x
```

```
        m = np.max(np.abs(x_new))
        x_new = x_new / m                    # Normalize
        if np.linalg.norm(x_new - x) < tol:
            return shift + 1.0 / m, x_new
        x = x_new
    return shift + 1.0 / m, x


# ------------------------------------------------
# MAIN -- Run all three methods and verify
# ------------------------------------------------
if __name__ == "__main__":

    # Example 3x3 symmetric matrix
    A = np.array([
        [4, 1, 0],
        [1, 3, 1],
        [0, 1, 2]
    ], dtype=float)

    print("Matrix A:")
    print(A)
    print()

    # 1. Largest eigenvalue
    lam_max, vec_max = power_method(A)
    print(f"Largest  Eigenvalue : {lam_max:.6f}")
    print(f"Eigenvector         : {vec_max}")
    print()

    # 2. Smallest eigenvalue
    lam_min, vec_min = inverse_power_method(A)
    print(f"Smallest Eigenvalue : {lam_min:.6f}")
    print(f"Eigenvector         : {vec_min}")
    print()

    # 3. Intermediate eigenvalue (shift close to but not
       equal to eigenvalue)
    shift_value = 2.5
    lam_mid, vec_mid = shifted_inverse_power_method(A,
       shift=shift_value)
    print(f"Intermediate Eigenvalue (shift={shift_value}):
       {lam_mid:.6f}")
    print(f"Eigenvector                                  : {
       vec_mid}")
    print()

    # 4. Verification using NumPy
    vals, _ = np.linalg.eig(A)
    print("NumPy Verification (all eigenvalues sorted):")
    print(np.sort(vals))
```

Listing 1: Complete Python implementation of all three eigenvalue methods

6.2 Code Explanation

Function	Purpose
`power_method(A)`	Finds $\lambda_{\max}$ using repeated $A\mathbf{x}$ multiplications
`inverse_power_method(A)`	Finds $\lambda_{\min}$ by applying Power Method to A^{-1}
`shifted_inverse_power_method`	Finds eigenvalue closest to a chosen shift σ
`np.linalg.inv(A)`	Computes the matrix inverse A^{-1}
`np.linalg.norm(...)`	Checks convergence via vector norm difference

6.3 Sample Output

```
Matrix A:
[[4. 1. 0.]
 [1. 3. 1.]
 [0. 1. 2.]]

Largest  Eigenvalue : 4.732053
Eigenvector         : [ 1.          0.73205214  0.26795017]

Smallest Eigenvalue : 1.267950
Eigenvector         : [ 0.26794896 -0.73205049  1.        ]

Intermediate Eigenvalue (shift=2.5): 3.000000
Eigenvector                        : [-0.99999989  0.99999985  1.        ]

NumPy Verification (all eigenvalues sorted):
[1.26794919  3.          4.73205081]
```

Verification: All three computed eigenvalues match the NumPy reference values exactly, confirming the implementation is correct.

7. Comparison of Methods

Feature	Power Method	Inverse Power	Shifted Inverse
Finds	Largest λ	Smallest λ	Any target λ
Operation	$A\mathbf{x}$	$A^{-1}\mathbf{x}$	$(A-\sigma I)^{-1}\mathbf{x}$
Setup cost	None	Compute A^{-1} once	Compute shifted A^{-1} once
Requires	$\lvert\lambda_n\rvert > \lvert\lambda_{n-1}\rvert$	$\lvert\lambda_1\rvert < \lvert\lambda_2\rvert$	Good shift σ
Convergence	Linear	Linear	Faster near σ

8. Applications

Field	Application	*Eigenvalue*
Structural Eng.	Critical buckling load of columns and beams	*Smallest*
Mechanical Eng.	Minimum natural frequency of vibrating structures	*Smallest*
Quantum Mechanics	Ground state energy of a quantum system	*Smallest*
Data Science	Principal Component Analysis (PCA)	*Largest*
Computer Graphics	Mesh simplification and shape analysis	*Intermediate*
Electrical Eng.	Stability of power systems and circuits	*Smallest*
Machine Learning	Spectral clustering and graph embeddings	*Smallest few*
Aeronautics	Flutter analysis of aircraft wings	*All*

9. Conclusion

The three iterative methods presented in this document form a complete toolkit for numerical eigenvalue computation:

- The **Power Method** is simple and reliable for finding the **dominant (largest)** eigenvalue. It requires no matrix inversion.

- The **Inverse Power Method** efficiently finds the **smallest eigenvalue** by applying the Power Method to A^{-1}. The key relationship is $\lambda_{\min}(A) = 1/\lambda_{\max}(A^{-1})$.
- The **Shifted Inverse Power Method** generalizes the approach to find **any intermediate eigenvalue** by shifting the matrix spectrum. A shift σ close to the target gives fast convergence.

Power Method: $\lambda_{\max} \approx m^{(k)}, \quad \mathbf{x}^{(k)} = \dfrac{A\,\mathbf{x}^{(k-1)}}{m^{(k)}}$

Inverse Power: $\lambda_{\min} = \dfrac{1}{m^{(k)}}, \quad \mathbf{x}^{(k)} = \dfrac{A^{-1}\mathbf{x}^{(k-1)}}{m^{(k)}}$

Shifted Inverse: $\lambda_{\text{target}} = \sigma + \dfrac{1}{m^{(k)}}, \quad \mathbf{x}^{(k)} = \dfrac{(A-\sigma I)^{-1}\mathbf{x}^{(k-1)}}{m^{(k)}}$

End of Document

Numerical Analysis – Inverse Power Method